

Database Systems

SQL Basics



SQL

- **SQL: Structured Query Language**
 - Data Definition Language (DDL)
The **Data-Definition** sublanguage for declaring database schemas
 - Data Manipulation Language (DML)
The **Data-Manipulation** sublanguage for querying databases and for modifying the database
- SQL
 - **Declarative** language
 - Supported by all major commercial database systems
 - Standardized: many new features over time
- SQL makes a distinction between
 - **Stored relations (Tables)**: Exists in DB and can be modified and queried
 - **Views** (Defined by Computation): Not stored but constructed when needed
 - **Temporary tables**: Constructed by SQL processor when it performs its job of executing queries and data modifications

SQL: Data Types

- Primitive Data Types

- Character string of fixed or varying Length (`CHAR(n)` , `VARCHAR(n)`)
- Bit Strings (`BIT VARYING(n)`)
- Boolean (`BOOLEAN`)
- Integer (`INT`, `INTEGER`, `SHORTINT`)
- Floating-point (`FLOAT`, `REAL`, `DOUBLE PRECISION`, `DECIMAL`, `NUMERIC`)
- Date and Time (`DATE` , `TIME`)

NOTE: SQL keywords (create and table for example) are not case sensitive.
Named objects (tables, columns etc.) *may* be.

SQL

✓ Create, Modify, Delete Table

- Insert Data Into Table
- Single-table Queries
 - The SFW query
 - Useful operators: DISTINCT, ORDER BY, LIKE
 - Handle missing values: NULLs
- Multiple-table Queries
 - Foreign key constraints
 - Joins: basics
 - Joins: SQL semantics
 - Set Operations
 - Inner/Outer Joins
 - Aggregation Queries & Subqueries

SQL: Create

- To create a table, use the **CREATE TABLE** statement
 - Specify the table name, field names and domains

```
CREATE TABLE Customer (  
    sin          CHAR(11) ,  
    firstName    CHAR(20) ,  
    lastName     CHAR(20) ,  
    age          INTEGER ,  
    income       REAL  
)
```

```
CREATE TABLE Movies(  
    title        CHAR(100) ,  
    year         INT ,  
    length       INT ,  
    genre        CHAR(10) ,  
    studioName   CHAR(30) ,  
    studioAddress CHAR(50)  
)
```

SQL: Modify Tables

- Modifying Relation Schemas
 - Delete a relation R: **DROP TABLE R;**
 - Modify Schema of relation R: **ALTER TABLE R**
 - **ADD** followed by an attribute name and its data type
 - **DROP** followed by an attribute name
- Example
 - ALTER TABLE Movies ADD producerCNum INT;**
 - ALTER TABLE Movies DROP studioAddress;**

SQL: DEFAULT

- Default Values
 - The value used when no other value is known
 - Keyword **DEFAULT**, Value either **NULL** or a Constant
 - Example

```
gender CHAR(1) DEFAULT '?',  
birthdate DATE DEFAULT DATE '0000-00-00',  
ALTER TABLE MovieStar ADD phone CHAR(16) DEFAULT 'unlisted';
```

SQL: Keys

- Declaring Keys
 - Declare key when attribute listed in the relation schema
 - Can be used only when the key is a single attribute
 - Or add to the list of items declared in schema an additional declaration that an attribute or set of attributes form the key
 - This method should be used if the key consists of more than one attribute
 - Two Declarations
 - Either **PRIMARY KEY**
 - Or **UNIQUE**
 - Set of attributes **S** key for relation **R**: Two tuples in **R** cannot agree on all of the attributes in set **S**, unless one of them is **NULL**. Violating action rejected.

SQL: Keys

- Primary Key Example

```
CREATE TABLE Movies (  
    title          CHAR(100) ,  
    year           INT ,  
    length         INT ,  
    genre          CHAR(10) ,  
    studioName     CHAR(30) ,  
    producerCNum   INT ,  
    PRIMARY KEY    (title, year)  
  
);
```

SQL

- Create, Modify, Delete Table

✓ Insert Data Into Table

- Single-table Queries
 - The SFW query
 - Useful operators: DISTINCT, ORDER BY, LIKE
 - Handle missing values: NULLs
- Multiple-table Queries
 - Foreign key constraints
 - Joins: basics
 - Joins: SQL semantics
 - Set Operations
 - Inner/Outer Joins
 - Aggregation Queries & Subqueries

SQL: Insert

- To insert a record into an existing table use the **INSERT** statement
 - The list of column names is optional
 - If omitted the values must be in the same order as the columns

```
INSERT INTO Customer(sin, firstName, lastName, age, income)
VALUES ('111', 'Sam', 'Spade', 23, 65234)
```

SQL: Modify Records

- Use the **UPDATE** statement to modify a record, or records, in a table
 - Note that the **WHERE** statement is evaluated *before* the **SET** statement
- Like **DELETE** the **WHERE** clause specifies which records are to be updated

```
UPDATE Customer  
SET age = 37  
WHERE sin = '111'
```

SQL: Delete

- To delete a record use the **DELETE** statement
 - The **WHERE** clause specifies the record(s) to be deleted

```
DELETE  
FROM Customer  
WHERE sin = '111'
```

- Be careful, the following SQL query deletes *all* the records in a table

```
DELETE  
FROM Customer
```

SQL

- Create, Modify, Delete Table
- Insert Data Into Table
- ✓ **Single-table Queries**
 - The SFW query
 - Useful operators: DISTINCT, ORDER BY, LIKE
 - Handle missing values: NULLs
- Multiple-table Queries
 - Foreign key constraints
 - Joins: basics
 - Joins: SQL semantics
 - Set Operations
 - Inner/Outer Joins
 - Aggregation Queries & Subqueries

Sample Table

Students

sid	name	school	age	gpa
1001	Adam	SFU	23	3.2
1002	Aiden	UBC	19	3.5
1003	Alice	SFU	18	3.7
1004	Bob	UBC	22	3.1
1005	David	SFU	20	3.2
1006	John	SFU	21	3.1
1007	Mary	UBC	21	3.4
1008	Mike	SFU	24	3.1
1009	Sarah	UBC	18	3.0

The SFW Query

- Select **F**rom **W**here
- To write the query, ask yourself three questions:
 - Which **table** do you want information from?
 - Which **rows** do you want information from?
 - Which **columns** do you want information from?

```
SELECT <columns>  
FROM   <table name>  
WHERE  <conditions>
```


Columns

- Which **columns** do you want information from?

SELECT * **FROM** **Students**



sid	name	school	age	gpa
1001	Adam	SFU	23	3.2
1002	Aiden	UBC	19	3.5
1003	Alice	SFU	18	3.7
1004	Bob	UBC	22	3.1
1005	David	SFU	20	3.2
1006	John	SFU	21	3.1
1007	Mary	UBC	21	3.4
1008	Mike	SFU	24	3.1
1009	Sarah	UBC	18	3.0

Columns

- Which **columns** do you want information from?

```
SELECT name, age  
FROM Students
```



sid	name	school	age	gpa
1001	Adam	SFU	23	3.2
1002	Aiden	UBC	19	3.5
1003	Alice	SFU	18	3.7
1004	Bob	UBC	22	3.1
1005	David	SFU	20	3.2
1006	John	SFU	21	3.1
1007	Mary	UBC	21	3.4
1008	Mike	SFU	24	3.1
1009	Sarah	UBC	18	3.0

Columns

- Which **columns** do you want information from?

```
SELECT name as studentName  
FROM Students
```



sid	name	school	age	gpa
1001	Adam	SFU	23	3.2
1002	Aiden	UBC	19	3.5
1003	Alice	SFU	18	3.7
1004	Bob	UBC	22	3.1
1005	David	SFU	20	3.2
1006	John	SFU	21	3.1
1007	Mary	UBC	21	3.4
1008	Mike	SFU	24	3.1
1009	Sarah	UBC	18	3.0

Columns

- Which **columns** do you want information from?

```
SELECT age * 365 as ageDay  
FROM Students
```



sid	name	school	age	gpa
1001	Adam	SFU	23	3.2
1002	Aiden	UBC	19	3.5
1003	Alice	SFU	18	3.7
1004	Bob	UBC	22	3.1
1005	David	SFU	20	3.2
1006	John	SFU	21	3.1
1007	Mary	UBC	21	3.4
1008	Mike	SFU	24	3.1
1009	Sarah	UBC	18	3.0

Conditions

- Which **rows** do you want information from?

WHERE gpa >= 3.5



sid	name	school	age	gpa
1001	Adam	SFU	23	3.2
1002	Aiden	UBC	19	3.5
1003	Alice	SFU	18	3.7
1004	Bob	UBC	22	3.1
1005	David	SFU	20	3.2
1006	John	SFU	21	3.1
1007	Mary	UBC	21	3.4
1008	Mike	SFU	24	3.1
1009	Sarah	UBC	18	3.0

Conditions

- Which **rows** do you want information from?

WHERE school='SFU'
AND gpa >= 3.5



sid	name	school	age	gpa
1001	Adam	SFU	23	3.2
1002	Aiden	UBC	19	3.5
1003	Alice	SFU	18	3.7
1004	Bob	UBC	22	3.1
1005	David	SFU	20	3.2
1006	John	SFU	21	3.1
1007	Mary	UBC	21	3.4
1008	Mike	SFU	24	3.1
1009	Sarah	UBC	18	3.0

Conditions

- Which **rows** do you want information from?

WHERE (school='SFU'
OR school='UBC')
AND gpa >= 3.5



sid	name	school	age	gpa
1001	Adam	SFU	23	3.2
1002	Aiden	UBC	19	3.5
1003	Alice	SFU	18	3.7
1004	Bob	UBC	22	3.1
1005	David	SFU	20	3.2
1006	John	SFU	21	3.1
1007	Mary	UBC	21	3.4
1008	Mike	SFU	24	3.1
1009	Sarah	UBC	18	3.0

Conditions

- Which **rows** do you want information from?

WHERE age * 365 > 7500

sid	name	school	age	gpa
1001	Adam	SFU	23	3.2
1002	Aiden	UBC	19	3.5
1003	Alice	SFU	18	3.7
1004	Bob	UBC	22	3.1
1005	David	SFU	20	3.2
1006	John	SFU	21	3.1
1007	Mary	UBC	21	3.4
1008	Mike	SFU	24	3.1
1009	Sarah	UBC	18	3.0

Values

- SQL **commands** are **CASE INSENSITIVE**
 - SELECT = Select = select
 - Student = student
 - gpa = GPA
- **Values** are **CASE SENSITIVE**
 - 'SFU' ≠ 'sfu'
- Quotation
 - SQL strings in **single quotes**
 - e.g. name = 'Mike'
 - Single quotes in a string can be specified using an initial single quote character as an escape
 - author = 'Shaq O"Neal'
 - Strings can be compared **alphabetically** with the comparison operators
 - e.g. 'fodder' < 'foo' is TRUE

Eliminating Repetitive Results

- **DISTINCT**: eliminating duplicates in the query result

SELECT school FROM Students

Versus

SELECT DISTINCT school FROM Students

school
SFU
UBC

school
SFU
UBC
SFU
UBC
SFU
SFU
UBC
SFU
UBC

Sorting the Results

- The output of an SQL query can be ordered using **ORDER BY**
 - By any number of attributes
 - In either ascending or descending order
- Default: **Ascending** order
- The keywords **ASC** and **DESC**, following the column name, set the order

Sorting the Results

```
SELECT    name, gpa, age
FROM      Students
WHERE     school = 'SFU'
ORDER BY  gpa DESC, age ASC
```

Simple String Pattern Matching

- SQL provides pattern matching support with the **LIKE** operator and two symbols
 - The **%** symbol stands for zero or more arbitrary characters
 - The **_** symbol stands for exactly one arbitrary character
 - The **%** and **_** characters can be escaped with ****
 - E.g., name **LIKE** 'Michael_Jordan'

```
SELECT * FROM Students WHERE name LIKE 'A_d%';
```

Simple String Pattern Matching

- Which names will be returned?

```
SELECT *  
FROM Students  
WHERE name LIKE 'Sm_t%'
```

1. Smit
2. SMIT
3. Smart
4. Smith
5. Smythe
6. Smut
7. Smeath
8. Smt

NULL

- Whenever we don't have a value, we can put a NULL
- Can mean many things:
 - Value does not exist
 - Value exists but is unknown
 - Value not applicable
 - Etc.
- NULL constraints

```
CREATE TABLE Students (  
    name CHAR(20) NOT NULL,  
    age CHAR(20) NOT NULL,  
    gpa FLOAT  
)
```

NULL: What will happen?

```
SELECT gpa*100 FROM students
```

```
SELECT name FROM students WHERE gpa > 3.5
```

```
SELECT name FROM students WHERE age > 19 OR gpa > 3.5
```

sid	name	school	age	gpa
1001	Adam	SFU	23	3.2
1002	Aiden	UBC	19	3.5
1003	Alice	SFU	18	3.7
1004	Bob	UBC	22	3.1
1005	David	SFU	20	NULL
1006	John	SFU	21	3.1
1007	Mary	UBC	21	3.4
1008	Mike	SFU	24	3.1
1009	Sarah	UBC	18	3.0

NULL: What will happen?

- Arithmetic operations (+, -, *, /) on nulls return **NULL**

NULL * 100 = NULL

NULL * 0 = NULL

- Comparisons with nulls evaluate to **UNKNOWN**

NULL > 3.5 → UNKNOWN

NULL = NULL → UNKNOWN

NULL: What will happen?

```
SELECT gpa*100 FROM students
```

```
SELECT gpa*0 FROM students
```

```
SELECT name FROM students WHERE gpa > 3.5
```

```
SELECT name FROM students WHERE gpa = NULL
```

Combinations of true, false, unknown

- Truth values for unknown results

TRUE OR UNKNOWN = **TRUE**

FALSE OR UNKNOWN = UNKNOWN

UNKNOWN OR UNKNOWN = UNKNOWN

TRUE AND UNKNOWN = UNKNOWN

FALSE AND UNKNOWN = **FALSE**

UNKNOWN AND UNKNOWN = UNKNOWN

Combinations of true, false, unknown

```
SELECT * FROM students WHERE age > 15 OR gpa > 3.5
```

```
SELECT * FROM students WHERE age > 15 AND gpa > 3.5
```

- The result of a **WHERE** clause is treated as **FALSE** if it evaluates to **UNKNOWN**
 - **WHERE UNKNOWN** → **FALSE**

Checking NULL Values

```
SELECT * FROM Students WHERE age < 25 OR age >= 25
```

```
SELECT * FROM Students  
WHERE age < 25 OR age >= 25 OR age IS NULL
```

- There are special operators to test for null values
 - IS NULL** tests for the presence of nulls and
 - IS NOT NULL** tests for the absence of nulls

SQL

- Create, Modify, Delete Table
- Insert Data Into Table
- Single-table Queries
 - The SFW query
 - Useful operators: DISTINCT, ORDER BY, LIKE
 - Handle missing values: NULLs

✓ Multiple-table Queries

- Foreign key constraints
- Joins: basics
- Joins: SQL semantics
- Set Operations
- Inner/Outer Joins
- Aggregation Queries & Subqueries

Foreign Key constraints

stid REFERENCES **sid**

Students

sid	name	school	age	gpa
1001	Adam	SFU	23	3.2
1002	Aiden	UBC	19	3.5
1003	Alice	SFU	18	3.7
1004	Bob	UBC	22	3.1
1005	David	SFU	20	3.2
1006	John	SFU	21	3.1
1007	Mary	UBC	21	3.4
1008	Mike	SFU	24	3.1
1009	Sarah	UBC	18	3.0

Enrolled

stid	cid	grade
1001	200	A
1001	295	A
1001	250	B+
1002	130	A
1002	125	A+
1003	120	A
1003	125	B
1003	150	B
1004	354	A

?



Foreign Key constraints

stid REFERENCES **sid**

Students

sid	name	school	age	gpa
1001	Adam	SFU	23	3.2
1002	Aiden	UBC	19	3.5
1003	Alice	SFU	18	3.7
1004	Bob	UBC	22	3.1
1006	John	SFU	21	3.1
1007	Mary	UBC	21	3.4
1008	Mike	SFU	24	3.1
1009	Sarah	UBC	18	3.0

Enrolled

stid	cid	grade
1001	200	A
1001	295	A
1001	250	B+
1002	130	A
1002	125	A+
1003	120	A
1003	125	B
1003	150	B
1005	354	A

←
Missing

Illegal

Declaring Foreign Key

```
CREATE TABLE Students (  
    sid          CHAR(20) ,  
    name         CHAR(20) ,  
    school       CHAR(15) ,  
    gpa          DOUBLE ,  
    PRIMARY KEY (sid)  
);
```

```
CREATE TABLE Enrolled(  
    stid  CHAR(20) ,  
    cid   CHAR(20) ,  
    grade CHAR(5) ,  
    PRIMARY KEY (stid, cid) ,  
    FOREIGN KEY (stid) REFERENCES  
    Students(sid)  
);
```

Foreign Key: Insert Operations

- What if we **insert** a tuple into Enrolled, but no corresponding student?
 - INSERT is rejected!**

Students

sid	name	school	age	gpa
1001	Adam	SFU	23	3.2
1002	Aiden	UBC	19	3.5
1003	Alice	SFU	18	3.7
1004	Bob	UBC	22	3.1
1006	John	SFU	21	3.1
1007	Mary	UBC	21	3.4
1008	Mike	SFU	24	3.1
1009	Sarah	UBC	18	3.0

Enrolled

stid	cid	grade
1001	200	A
1001	295	A
1001	250	B+
1002	130	A
1002	125	A+
1003	120	A
1003	125	B
1003	150	B
1015	371	A

Illegal

Foreign Key: Delete Operations

- What if we delete a student, who has enrolled courses?
 - Options
 - Disallow the delete
 - **ON DELETE RESTRICT**
 - Remove all of the courses for that student
 - **ON DELETE CASCADE**
 - Set Foreign Key to NULL
 - **ON DELETE SET NULL**

Foreign Key: Delete Operations: Restrict

```
CREATE TABLE Enrolled(  
    stid          CHAR(20),  
    cid           CHAR(20),  
    grade         CHAR(10),  
    PRIMARY KEY  (stid, cid),  
    FOREIGN KEY  (stid)  
    REFERENCES  Students(sid)  
    ON DELETE RESTRICT  
)
```

Foreign Key: Delete Operations: Restrict

Students

sid	name	school	age	gpa
1001	Adam	SFU	23	3.2
1002	Aiden	UBC	19	3.5
1003	Alice	SFU	18	3.7
1004	Bob	UBC	22	3.1
1006	John	SFU	21	3.1
1007	Mary	UBC	21	3.4
1008	Mike	SFU	24	3.1
1009	Sarah	UBC	18	3.0

**Delete Operation
on Students
Not Allowed!**

Enrolled

stid	cid	grade
1001	200	A
1001	295	A
1001	250	B+
1002	130	A
1002	125	A+
1003	120	A
1003	125	B
1003	150	B

**Because sid referenced by stid
and ON DELETE RESTRICT**

Foreign Key: Delete Operations: Cascade

```
CREATE TABLE Enrolled(  
    std          CHAR(20) ,  
    cid          CHAR(20) ,  
    grade       CHAR(10) ,  
    PRIMARY KEY (std, cid) ,  
    FOREIGN KEY (std)  
    REFERENCES Students(sid)  
    ON DELETE CASCADE  
)
```

Foreign Key: Delete Operations: Cascade

Students

sid	name	school	age	gpa
1001	Adam	SFU	23	3.2
1002	Aiden	UBC	19	3.5
1003	Alice	SFU	18	3.7
1004	Bob	UBC	22	3.1
1006	John	SFU	21	3.1
1007	Mary	UBC	21	3.4
1008	Mike	SFU	24	3.1
1009	Sarah	UBC	18	3.0

**Delete Operation
on Students
causes delete
rows with same
sids in stdid
In Enrolled**

Enrolled

stdid	cid	grade
1001	200	A
1001	295	A
1001	250	B+
1002	130	A
1002	125	A+
1003	120	A
1003	125	B
1003	150	B

**Because sid referenced by stdid
and **ON DELETE CASCADE****

Foreign Key: Delete Operations: Set NULL

```
CREATE TABLE Enrolled(  
    stid          CHAR(20) ,  
    cid          CHAR(20) ,  
    grade        CHAR(10) ,  
    PRIMARY KEY  (stid, cid) ,  
    FOREIGN KEY  (stid)  
    REFERENCES  Students(sid)  
    ON DELETE SET NULL  
)
```


Foreign Key: Delete Operations: Set NULL

Students

sid	name	school	age	gpa
1001	Adam	SFU	23	3.2
1002	Aiden	UBC	19	3.5
1003	Alice	SFU	18	3.7
1004	Bob	UBC	22	3.1
1006	John	SFU	21	3.1
1007	Mary	UBC	21	3.4
1008	Mike	SFU	24	3.1
1009	Sarah	UBC	18	3.0

Delete Operation
on Students
causes NULL
values in sid on
rows with same
sids in sid
In Enrolled. Still
Rejected!

Enrolled

stid	cid	grade
1001	200	A
1001	295	A
1001	250	B+
1002	130	A
1002	125	A+
NULL	120	A
NULL	125	B
NULL	150	B

Because sid referenced by stid
and **ON DELETE SET NULL**

Although satisfies the foreign-key constraint
violates the **primary-key constraint**
thus deletion operation is rejected

SQL

- Create, Modify, Delete Table
- Insert Data Into Table
- Single-table Queries
 - The SFW query
 - Useful operators: DISTINCT, ORDER BY, LIKE
 - Handle missing values: NULLs

✓ Multiple-table Queries

- Foreign key constraints
- **Joins: basics**
- Joins: SQL semantics
- Set Operations
- Inner/Outer Joins
- Aggregation Queries & Subqueries

Joins

- Single Table
 - **WHERE Conditions**
 - Which **rows** do you want information from?
- Multiple Tables
 - **WHERE Conditions**
 - Which **rows** do you want information from?
 - How to join multiple tables

Join

- Find the names of students with an A+ in 125

```
SELECT name
FROM Students, Enrolled
WHERE sid = stid AND ← Joining Tables
      cid = 125 AND grade = 'A+' ← Rows
```

Join: Other ways to write the same

```
SELECT name
FROM Students
JOIN Enrolled ON sid = stid
WHERE cid = 125 AND grade = 'A+'
```

```
SELECT name
FROM Students
JOIN Enrolled ON sid = stid
AND cid = 125 AND grade = 'A+'
```

Tuple Variable

```
SELECT name  
FROM Students, Enrolled
```

- What if both tables have an attribute name!?

```
SELECT S.name  
FROM Students S, Enrolled
```

```
SELECT Students.name  
FROM Students, Enrolled
```

SQL

- Create, Modify, Delete Table
- Insert Data Into Table
- Single-table Queries
 - The SFW query
 - Useful operators: DISTINCT, ORDER BY, LIKE
 - Handle missing values: NULLs

✓ Multiple-table Queries

- Foreign key constraints
- Joins: Basics
- **Joins: SQL semantics**
- Set Operations
- Inner/Outer Joins
- Aggregation Queries & Subqueries

Joins: SQL Semantics

For each tuple r in R do

 For each tuple s in S do

 If r and s satisfy the join condition

 Then output the tuple $\langle r, s \rangle$

This is called **nested loop semantics** since we are interpreting what a join means using a nested loop

Joins: SQL Semantics

```
SELECT r1.s1, r1.s2, ..., rn.sk
FROM   R1 AS r1, R2 AS r2, ..., Rn AS rn
WHERE  Conditions(r1, ..., rn)
```

```
Answer = {}
for r1 in R1 do
  for r2 in R2 do
    ...
    for rn in Rn do
      if Conditions(r1, ..., rn)
        then Answer = Answer ∪ { (r1.s1, r1.s2, ..., rn.sk) }
return Answer
```

SQL

- Create, Modify, Delete Table
- Insert Data Into Table
- Single-table Queries
 - The SFW query
 - Useful operators: DISTINCT, ORDER BY, LIKE
 - Handle missing values: NULLs

✓ Multiple-table Queries

- Foreign key constraints
- Joins: basics
- Joins: SQL semantics
- **Set Operations**
- Inner/Outer Joins
- Aggregation Queries & Subqueries

Set Operations

- SQL supports union, intersection and set difference operations
 - Called **UNION**, **INTERSECT**, and **EXCEPT**
 - These operations must be performed on **union compatible tables**
 - Two **tables** are **union compatible** if
 - Have same number of attributes
 - Corresponding attributes have same data types
- Although these operations are supported in the SQL standard, **implementations may vary**
 - **EXCEPT**
 - May not be implemented
 - May be implemented as **MINUS**

Set Operations

```
SELECT name
FROM   Students S, Enrolled E1, Enrolled E2
WHERE  S.sid = E1.stid AND S.sid = E2.stid
      AND (E1.cid = 120 AND E2.cid = 125)
```



Set Operations: Union

- Find student names who have taken **120** or **125**.

sid	name	school	age	gpa
1001	Adam	SFU	23	3.2
1002	Aiden	UBC	19	3.5
1003	Alice	SFU	18	3.7
1004	Bob	UBC	22	3.1
1006	John	SFU	21	3.1
1007	Mary	UBC	21	3.4
1008	Mike	SFU	24	3.1
1009	Sarah	UBC	18	3.0

stid	cid	grade
1001	200	A
1001	295	A
1001	250	B+
1002	130	A
1002	125	A+
1003	120	A
1003	125	B
1003	150	B

Set Operations: Union

```
SELECT name
FROM Students, Enrolled
WHERE sid = stid AND cid = 120
```

UNION

```
SELECT name
FROM Students, Enrolled
WHERE sid = stid AND cid = 125
```

Set Operations: Intersect

- Find student names who have taken both of 120 and 125.

sid	name	school	age	gpa
1001	Adam	SFU	23	3.2
1002	Aiden	UBC	19	3.5
1003	Alice	SFU	18	3.7
1004	Bob	UBC	22	3.1
1006	John	SFU	21	3.1
1007	Mary	UBC	21	3.4
1008	Mike	SFU	24	3.1
1009	Sarah	UBC	18	3.0

stid	cid	grade
1001	200	A
1001	295	A
1001	250	B+
1002	130	A
1002	125	A+
1003	120	A
1003	125	B
1003	150	B

Set Operations: Intersect

```
SELECT name  
FROM Students, Enrolled  
WHERE sid = stid AND cid = 120
```

INTERSECT

```
SELECT name  
FROM Students, Enrolled  
WHERE sid = stid AND cid = 125
```


Set Operations: Except

- Find student names who have taken 125 but not 120.

sid	name	school	age	gpa
1001	Adam	SFU	23	3.2
1002	Aiden	UBC	19	3.5
1003	Alice	SFU	18	3.7
1004	Bob	UBC	22	3.1
1006	John	SFU	21	3.1
1007	Mary	UBC	21	3.4
1008	Mike	SFU	24	3.1
1009	Sarah	UBC	18	3.0

stid	cid	grade
1001	200	A
1001	295	A
1001	250	B+
1002	130	A
1002	125	A+
1003	120	A
1003	125	B
1003	150	B

Set Operations: Except

```
SELECT name  
FROM Students, Enrolled  
WHERE sid = stid AND cid = 125
```

EXCEPT

```
SELECT name  
FROM Students, Enrolled  
WHERE sid = stid AND cid = 120
```

Set Operations: Duplicates

- Set operation queries **eliminate duplicates** by default
 - **UNION**, **INTERSECT**, and **EXCEPT**
 - Unlike other SQL operations
- SQL allows duplicates to be retained in these three operations using the **ALL** keyword

Set Operations: Keep Duplicates

```
SELECT name  
FROM Students, Enrolled  
WHERE sid = stid AND cid = 120
```

INTERSECT ALL

```
SELECT name  
FROM Students, Enrolled  
WHERE sid = stid AND cid = 125
```

SQL

- Create, Modify, Delete Table
- Insert Data Into Table
- Single-table Queries
 - The SFW query
 - Useful operators: DISTINCT, ORDER BY, LIKE
 - Handle missing values: NULLs

✓ Multiple-table Queries

- Foreign key constraints
- Joins: basics
- Joins: SQL semantics
- Set Operations
- **Inner/Outer Joins**
- Aggregation Queries & Subqueries

Example Tables

Students

sid	name	school	age	gpa
1001	Adam	SFU	23	3.2
1002	Aiden	UBC	19	3.5
1003	Alice	SFU	18	3.7
1004	Bob	UBC	22	3.1
1006	John	SFU	21	3.1
1007	Mary	UBC	21	3.4
1008	Mike	SFU	24	3.1
1009	Sarah	UBC	18	3.0

Enrolled

stid	cid	grade
1001	200	A
1001	295	A
1001	250	B+
1002	130	A
1002	125	A+
1003	120	A
1003	125	B
1003	150	B

Joins

```
SELECT name, cid
FROM   Students INNER JOIN Enrolled ON sid = stid;
```

```
SELECT name, cid
FROM   Students FULL OUTER JOIN Enrolled ON sid = stid;
```

```
SELECT name, cid
FROM   Students LEFT OUTER JOIN Enrolled ON sid = stid;
```

```
SELECT name, cid
FROM   Students RIGHT OUTER JOIN Enrolled ON sid = stid;
```

Behavior on unmatched: Outer (on the given table) retains unmatched tuples (from that table) with NULL

SQL

- Create, Modify, Delete Table
- Insert Data Into Table
- Single-table Queries
 - The SFW query
 - Useful operators: DISTINCT, ORDER BY, LIKE
 - Handle missing values: NULLs

✓ Multiple-table Queries

- Foreign key constraints
- Joins: basics
- Joins: SQL semantics
- Set Operations
- Inner/Outer Joins
- **Aggregation Queries & Subqueries**

Simple Aggregation

```
SELECT AGG (column)
FROM   <table name>
WHERE  <conditions>
```

AGG = COUNT, SUM, AVG, MAX, MIN, etc

- Count applies to more than one attribute
- Other aggregations apply to a single attribute

Examples: Simple Aggregation

- `SELECT COUNT (*) FROM Students;`
- `SELECT SUM(gpa) FROM Students;`
- `SELECT AVG(gpa) FROM Students;`
- `SELECT MIN(gpa) FROM Students;`
- `SELECT MAX(gpa) FROM Students;`

Examples: Simple Aggregation

- `SELECT COUNT(DISTINCT gpa) FROM Students;`
- `SELECT SUM(DISTINCT gpa) FROM Students;`
- `SELECT AVG(gpa) FROM Students WHERE age = 19;`

Group By: The Need

- How to get AVG(gpa) for each age?

```
SELECT AVG(gpa) FROM Students WHERE age = 18;
```

```
SELECT AVG(gpa) FROM Students WHERE age = 19;
```

```
SELECT AVG(gpa) FROM Students WHERE age = 20;
```

```
SELECT AVG(gpa) FROM Students WHERE age = 21;
```

Grouping & Aggregation

```
SELECT agg(column)
FROM <table name>
WHERE <conditions>
GROUP BY <columns>
```

- How to get AVG(gpa) for each age?
`SELECT AVG(gpa) FROM Students GROUP BY age;`

Grouping

- How is the following query processed?

```
SELECT age, AVG(gpa)
FROM Students
WHERE gpa > 2.5
GROUP BY age;
```

- Semantics of the query
 - Compute the **FROM** and **WHERE** clauses
 - Group by the attributes in the **GROUP BY**
 - Compute the **SELECT** clause: grouped attributes and aggregates

Grouping

- Special Cases
 - Selecting Attributes
 - Everything in **SELECT** must be either a GROUP-BY attribute, or an aggregate
 - Empty Groups

Having

- Specify which groups you are interested in

```
SELECT agg(column)
FROM    <table name>
WHERE   <conditions>
GROUP BY <columns>
HAVING  <columns>
```

- **HAVING** clause contains conditions on aggregates.

Example: Having

```
SELECT AVG(gpa) , age
FROM Students
WHERE gpa > 2.5
GROUP BY age
HAVING COUNT(*) >= 2;
```

Having: Order of Evaluation

- Create the cross product of the tables in the **FROM** clause
- Remove rows not meeting the **WHERE** condition
- Divide records into groups by the **GROUP BY** clause
- Remove groups not meeting the **HAVING** clause
- Create one row for each group and remove columns not in the **SELECT** clause

Subqueries

✓ Subqueries

- **FROM** clause subqueries
- **WHERE** clause subqueries

Subqueries

- Need to compute an intermediate table only to use it later in a **SFW**
- Subqueries may appear in
 - A **WHERE** clause
 - Subqueries can return a single constant, and this constant can be compared with another value in a **WHERE** clause.
 - Subqueries can return relations that can be used in various ways in **WHERE** clauses.
 - A **FROM** clause
 - Subqueries can appear in **FROM** clauses, followed by a tuple variable that represents the tuples in the result of the subquery.
- Subqueries may have subqueries, down as many levels as we wish.

Subqueries

- Subquery: a query that is part of another query
- Such inner-outer queries are called nested queries

```
SELECT name
FROM MovieExec
WHERE certNum =
    ( SELECT producerCNum
      FROM Movies
      WHERE title = 'Star Wars'
    );
```

} Outer Query

} Inner Query

Subqueries in WHERE Clauses

- Subqueries that Produce Scalar Values
- Conditions Involving Relations
- Conditions Involving Tuples
- Correlated Subqueries

Subqueries that Produce Scalar Values

- Scalar: atomic value that can be one component of a tuple
- Comparing the result of a subquery (which is a scalar value) to a constant or attribute in **WHERE** clause

Example: Subqueries that Produce Scalar Values

```
Movies(title, year, length, genre, studioName, producerCNum) StarsIn(movieTitle,  
movieYear, starName)
```

```
MovieExec(name, address, certNum, netWorth)
```

```
SELECT name  
FROM Movies, MovieExec  
WHERE title = 'Star Wars' AND producerCNum = certNum;
```

```
SELECT name FROM MovieExec WHERE certNum =  
    (  
        SELECT producerCNum  
        FROM Movies  
        WHERE title = 'Star Wars'  
    );
```


Conditions Involving Relations

- A relation **R** must be expressed as a subquery.

IN

NOT IN

EXISTS

NOT EXISTS

ANY

ALL

Conditions Involving Relations

- **EXISTS R**: true if and only if **R** is not empty.
- **s IN R**: true if and only if **s** is equal to one of the values in unary relation **R**.
- **s NOT IN R**: true if and only if **s** is equal to no value in unary relation **R**.
- **s > ALL R**: true if and only if **s** is greater than every value in unary relation **R**.
- **s > ANY R**: true if and only if **s** is greater than at least one value in unary relation **R**.

Conditions Involving Tuples

- If a tuple **t** has the same number of components as a relation **R**, then it makes sense to compare **t** and **R** in expressions.

```
SELECT name
FROM MovieExec
WHERE certNum IN
    (
        SELECT producerCNum
        FROM Movies
        WHERE (title, year) IN
            (
                SELECT movieTitle, movieYear
                FROM StarsIn
                WHERE starName = 'Harrison Ford'
            )
    )
;
```

Correlated Subqueries

- A subquery to be evaluated many times, once for each assignment of a value to some term in the subquery that comes from a tuple variable outside the subquery.

- Example: Finding movie titles that appear more than once

```
SELECT title
FROM Movies Old
WHERE year < ANY
    (
        SELECT year
        FROM Movies
        WHERE title = Old.title
    );
```

Subqueries in FROM Clauses

- In a FROM list, instead of a stored relation, we may use a parenthesized subquery.

```
SELECT name
FROM MovieExec, ( SELECT producerCNum
                   FROM Movies, StarsIn
                   WHERE title = movieTitle AND
                        year = movieYear AND
                        starName = 'Harrison Ford'
                   ) as Prod
WHERE certNum = Prod.producerCNum;
```

Acknowledgements

- [1] CMPT 354 (Jiannan Wang - SFU)
<https://sfu-db.github.io/cmpt354/>

- [2] Database Systems: The Complete Book, 2nd Edition
Hector Garcia-Molina, Jeffrey D. Ullman, Jennifer Widom
Prentice Hall, 2009

- [3] Database System Concepts, Seventh Edition
Avi Silberschatz, Henry F. Korth, S. Sudarshan
McGraw-Hill, March 2019
www.db-book.com

Additional references and resources used in preparation of this course are listed on <https://canvas.sfu.ca/courses/13083/pages/references> or mentioned on each slide.